

第5章 优化程序性能

本章讨论如何写出高效程序。优化程序性能不是简单地“写得更短”，而是理解程序的瓶颈：重复计算、函数调用开销、内存别名、数据依赖、分支预测失败、Cache不命中等。掌握本章后，应能看到一段循环程序并判断它慢在哪里，以及该用哪类优化方法。

1. 优化的基本层次

程序优化可以分为三个层次：

层次一：与机器无关的优化（编译器能做或应该做的）

- 代码移动
- 共享公用子表达式
- 复杂运算简化
- 消除不必要的函数调用和内存引用

层次二：面向编译器的优化（帮助编译器生成更好的代码）

- 使用局部变量替代全局变量或指针间接访问
- 使用内联函数（`inline`）避免函数调用开销

层次三：面向现代CPU的优化（利用超标量和流水线）

- 循环展开
- 分离的累加器
- 向量处理指令（SIMD/AVX）

学习本章时要始终记住：

- 优化首先要保证程序语义正确。
- 优化应优先针对程序最耗时的部分，通常是内层循环。
- 编译器很强，但遇到函数副作用、内存别名、浮点舍入等问题时会保守处理。

2. 编译器优化的能力与限制

(1) 编译器能做什么

编译器可以自动做许多优化：

- 删除无用代码。

- 将常量表达式提前计算。
- 将乘以常数转化为移位和加法。
- 对简单循环做一定程度的展开。
- 对明显重复的表达式做公共子表达式消除。

例如：

```
x = 16 * y;
```

编译器可能将其转为：

```
x = y << 4;
```

因为左移通常比乘法更便宜。

(2) 限制一：函数调用副作用

看下面的代码：

```
void lower(char *s) {  
    for (int i = 0; i < strlen(s); i++) {  
        if (s[i] >= 'A' && s[i] <= 'Z') {  
            s[i] -= 'A' - 'a';  
        }  
    }  
}
```

`strlen(s)` 看起来每次结果都一样，但编译器不一定敢把它移出循环，因为函数调用可能读取或修改内存状态。手动优化如下：

```
void lower_opt(char *s) {
    int len = strlen(s);
    for (int i = 0; i < len; i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') {
            s[i] -= 'A' - 'a';
        }
    }
}
```

优化思想：

- `strlen(s)` 是循环不变量。
- 将它移出循环后，避免每轮重复扫描字符串。
- 程序复杂度可能从 $O(n^2)$ 降到 $O(n)$ 。

(3) 限制二：内存别名

内存别名指两个指针可能指向同一个内存位置。例如：

```
void twiddle1(long *xp, long *yp) {
    *xp += *yp;
    *xp += *yp;
}
```

如果 `xp != yp`，似乎可以优化成：

```
*xp += 2 * (*yp);
```

但如果 `xp == yp`，原程序行为是：

```
第一次：*xp = *xp + *xp = 2x
第二次：*xp = 2x + 2x = 4x
```

而优化后是：

```
*xp = x + 2x = 3x
```

结果不同。因此编译器必须保守。常用优化方法是把反复使用的值放入局部变量：

```
void sum_rows(double *a, double *b, long n) {
    for (long i = 0; i < n; i++) {
        double val = 0;
        for (long j = 0; j < n; j++) {
            val += a[i*n + j];
        }
        b[i] = val;
    }
}
```

`val` 通常可保存在寄存器中，避免每次循环都读写 `b[i]`。

3. 表示程序性能：CPE

CPE（Cycles Per Element）表示处理每个元素所需的时钟周期数，是衡量循环性能的重要指标。

如果处理 `n` 个元素的总周期数近似为：

$$\text{cycles} = a \times n + b$$

则 `a` 就是CPE，`b` 是固定开销。CPE越小，说明单位元素处理越快。

示例：

n	总周期数
1000	5200
2000	10200
3000	15200

每增加1000个元素，多约5000周期，所以CPE约为5。

4. 常见基本优化方法

(1) 代码移动

代码移动是把循环中不变的计算移出循环。

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        a[n*i + j] = b[j];
    }
}
```

`n*i` 在内层循环中不变，可改为：

```
for (int i = 0; i < n; i++) {
    int ni = n * i;
    for (int j = 0; j < n; j++) {
        a[ni + j] = b[j];
    }
}
```

(2) 公共子表达式消除

二维数组访问会隐含乘法和加法：

`A[i][j]` 等价于 `*(A + i*COLS + j)`

对于邻域计算：

```
sum = A[i-1][j] + A[i+1][j] + A[i][j-1] + A[i][j+1];
```

先计算中心位置：

```
long center = i * COLS + j;
sum = data[center - COLS] + data[center + COLS]
      + data[center - 1]    + data[center + 1];
```

这类方法常用于图像平滑、矩阵模板、卷积等程序。

(3) 强度削弱

强度削弱是用便宜运算替代昂贵运算。例如：

```
x * 16  → x << 4
x / 2   → x >> 1
```

循环中的乘法也可以改成累加：

```
int ni = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        a[ni + j] = b[j];
    }
    ni += n;
}
```

5. 现代处理器与性能界限

现代处理器具有流水线、超标量和乱序执行能力。

- **流水线**：不同指令的不同阶段同时进行。
- **超标量**：一个周期可发射多条指令。
- **乱序执行**：处理器在保证最终结果正确的前提下，动态调整指令执行顺序。

(1) 延迟、发射时间、容量

参数	含义
延迟	完成一次操作需要的总周期数
发射时间	两次同类操作开始之间的最小间隔
容量	能执行该操作的功能单元数量

(2) 延迟界限

如果循环中存在串行依赖：

```
for (int i = 0; i < n; i++) {
    acc = acc * data[i];
}
```

每次迭代都依赖上一次 `acc` 的结果。如果浮点乘法延迟为5周期，则CPE很可能接近5。

(3) 吞吐量界限

如果处理器有多个乘法单元，并且每周期都能开始新操作，理论上可以同时推进多条独立运算链。吞吐量界限由功能单元能力决定：

$$CPE \geq \text{发射时间} / \text{容量}$$

这个式子是简化管理：假设每个元素主要只需要一次该类操作，且没有加载、存储、分支等其他瓶颈。更一般地，如果程序一共需要 N 次某类操作，功能单元容量为 C ，发射时间为 I ，那么至少需要：

$$\text{总周期数} \geq N \times I / C$$

实际性能通常受下面两者较大者限制：

$$\text{实际CPE} \approx \max(\text{延迟界限}, \text{吞吐量界限})$$

6. 利用指令级并行

(1) 循环展开

循环展开减少循环控制开销：

```
for (int i = 0; i < limit; i += 2) {  
    acc = (acc OP data[i]) OP data[i+1];  
}
```

但如果仍只有一个 `acc`，数据依赖链没有断，性能提升有限。

(2) 多个累加器

使用多个独立累加器：

```
for (int i = 0; i < limit; i += 2) {
    acc0 = acc0 OP data[i];
    acc1 = acc1 OP data[i+1];
}
result = acc0 OP acc1;
```

优点：

- `acc0` 和 `acc1` 互不依赖。
- 处理器可并行推进多条运算链。
- 能更接近吞吐量界限。

(3) 重新结合

```
acc = (acc OP data[i]) OP data[i+1];
```

可改成：

```
acc = acc OP (data[i] OP data[i+1]);
```

这样 `data[i] OP data[i+1]` 能够提前计算，从而缩短关键路径。

注意：浮点运算受舍入影响，不严格满足结合律，重排可能导致末尾位不同。

7. SIMD 向量化

SIMD（单指令多数据）用一条指令同时处理多个数据。例如256位YMM寄存器可同时处理：

数据类型	可同时处理数量
float 32位	8个
double 64位	4个
int 32位	8个

适合场景：

- 大数组逐元素相同操作。

- 图像处理。
- 矩阵计算。
- 数值计算。

8. 分支预测与条件传送

分支预测错误会清空流水线，代价较高。对于简单选择，可以使用条件传送。

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

条件传送适合：

- 两个分支计算都简单。
- 没有副作用。
- 分支方向难预测。

不适合：

- 某个分支可能非法访问。
- 某个分支计算代价很大。
- 分支包含函数调用或写内存。

9. 面向存储器的优化

Cache友好代码的基本原则：

- 让内层循环步长为1。
- 按内存连续顺序访问数组。
- 用局部变量减少重复读写内存。
- 对大矩阵使用分块。

矩阵分块示例：

```
for (int ii = 0; ii < N; ii += B) {
    for (int jj = 0; jj < N; jj += B) {
        for (int i = ii; i < ii + B && i < N; i++) {
            for (int j = jj; j < jj + B && j < N; j++) {
                use(A[i][j]);
            }
        }
    }
}
```

分块的目标是让当前工作集放入Cache，增加块内数据复用。

第6章 存储器层次结构

本章解释计算机为什么要设计多级存储器，以及Cache如何利用程序局部性提升性能。掌握本章后，应能根据Cache参数拆分地址、分析命中和缺失、解释存储器山、编写Cache友好代码。

1. 存储器层次结构

存储器从快到慢、从小到大大致为：

寄存器 → L1 Cache → L2 Cache → L3 Cache → 主存 → 外部存储

基本规律：

- 越靠近CPU，速度越快，容量越小，单位成本越高。
- 上层存储器作为下层存储器的缓存。
- 局部性越好，缓存效果越明显。

2. 局部性

局部性是Cache有效的根本原因。

类型	含义	示例
时间局部性	最近访问过的数据很快再次访问	循环变量、重复使用数组元素
空间局部性	访问一个位置后很快访问相邻位置	顺序扫描数组

数组顺序访问：

```
for (int i = 0; i < n; i++) {  
    sum += a[i];  
}
```

空间局部性好，因为 `a[i]`、`a[i+1]`、`a[i+2]` 在内存中连续。

链表遍历：

```
for (node *p = head; p != NULL; p = p->next) {  
    sum += p->val;  
}
```

空间局部性较差，因为节点可能分散在内存各处。

3. Cache基本组织

Cache由组、行、块组成。

符号	含义
C	Cache总容量
B	每个块的字节数
E	每组的行数，相联度
S	组数

基本公式：

```
C = S × E × B  
S = C / (E × B)
```

一个地址被分成三部分：

```
| tag | set index | block offset |
```

位数计算：

```
b = log2(B)
s = log2(S)
t = m - s - b
```

其中 m 是地址位数。

4. Cache地址划分示例

假设：

```
C = 32KB
E = 4
B = 64B
地址位数m = 32
```

计算：

```
S = 32KB / (4 × 64B)
  = 32768 / 256
  = 128组

b = log2(64) = 6
s = log2(128) = 7
t = 32 - 7 - 6 = 19
```

地址格式：

```
| tag 19位 | set index 7位 | block offset 6位 |
```

注意：

- 块内偏移单位是字节，不是数组元素。
- 数组元素地址要先乘以元素大小。

5. 直接映射、组相联、全相联

类型	E	特点
直接映射	1	每个块只能放入一个组的一行，简单但冲突多
组相联	大于1	每个组有多行，冲突较少
全相联	只有1组	块可放任意行，冲突少但硬件复杂

组相联通常减少冲突不命中，但不表示在所有访问模式下一定优于直接映射。

6. Cache缺失类型

类型	原因	例子
冷(强制)不命中	第一次访问某块	第一次读数组第一个元素
冲突不命中	多个块映射到同一组，互相替换	直接映射中两个数组地址冲突
容量不命中	工作集大于Cache容量	大矩阵一次性扫描

7. 不命中率与平均访问时间

平均访问时间公式：

$$\text{平均访问时间} = \text{命中时间} + \text{不命中率} \times \text{不命中惩罚}$$

例如命中时间2周期，不命中惩罚100周期，不命中率5%：

$$\text{平均访问时间} = 2 + 0.05 \times 100 = 7 \text{周期}$$

顺序访问 `int` 数组，若Cache块为64B：

$$\text{每块可放 } 64 / 4 = 16 \text{个int}$$

如果只读数组，理想情况下大约每16次访问出现1次块不命中。

8. 存储器山

存储器山用两个参数观察程序访问性能：

参数	反映内容
size	工作集大小，反映时间局部性
stride	访问步长，反映空间局部性

结论：

- `stride=1` 时，Cache块内数据利用率高，性能好。
- `stride` 变大时，一个Cache块中很多数据被浪费，性能下降。
- `size` 超过L1、L2、L3容量时，性能会逐级下降。

9. 编写Cache友好代码

(1) 内层循环步长为1

C语言二维数组按行优先存储：

```
int A[100][100];
```

访问 `A[i][j]` 时，`j` 连续变化更友好：

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        sum += A[i][j];
    }
}
```

(2) 使用局部变量

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += a[i];
}
```

`sum` 可保存在寄存器中，减少内存访问。

(3) 分块

对矩阵运算：

```
for (int ii = 0; ii < N; ii += B) {
    for (int jj = 0; jj < N; jj += B) {
        for (int i = ii; i < ii+B; i++) {
            for (int j = jj; j < jj+B; j++) {
                use(A[i][j]);
            }
        }
    }
}
```

分块思想：让小块数据在Cache中被多次复用。

第8章 异常控制流

本章讨论程序控制流如何被异常、进程、信号和系统调用改变。理解本章后，应能解释系统调用如何进入内核、`fork` 后程序如何分裂、`execve` 如何加载新程序、Shell如何回收子进程。

1. 异常控制流的概念

正常控制流是指程序按指令顺序执行。异常控制流是指由于某种事件，控制流跳转到其他位置，例如内核异常处理程序或信号处理程序。

常见事件：

- 键盘输入。
- 系统调用。
- 缺页。
- 除以0。
- 子进程终止。

2. 异常四分类

类型	原因	同步/异步	返回行为	例子
中断	外部I/O设备	异步	下一条指令	键盘、定时器
陷阱	程序主动请求内核	同步	下一条指令	系统调用

故障	当前指令造成可恢复问题	同步	当前指令或终止	缺页
终止	不可恢复错误	同步	不返回	严重硬件错误

记忆方式：

外设来的是中断；主动进内核是陷阱；
能修再来是故障；修不了就是终止。

3. 进程的两个抽象

进程是运行中程序的实例。它提供两个关键抽象：

- 1. **独立逻辑控制流**：程序好像独占CPU。
- 2. **私有地址空间**：程序好像独占内存。

实际上：

- CPU通过上下文切换在多个进程之间共享。
- 内存通过页表为进程提供独立虚拟地址空间。

4. fork：创建子进程

```
pid_t pid = fork();
```

返回值：

位置	返回值
子进程	0
父进程	子进程PID
出错	-1

核心结论：

fork调用一次，返回两次

示例：

```
printf("A\n");  
fork();  
printf("B\n");
```

结果：

- A 打印1次。
- B 打印2次。

两个连续 fork：

```
fork();  
fork();  
printf("X\n");
```

进程数变化：

1 → 2 → 4

所以 X 打印4次。

注意：父子进程执行顺序不确定，除非程序使用 `waitpid` 等机制同步。

5. `execve`：加载并运行新程序

```
int execve(char *filename, char *argv[], char *envp[]);
```

核心结论：

- `execve` 不创建新进程。
- 它在当前进程中加载新程序。
- 成功时不返回。
- PID不变，但用户地址空间被替换。

与 `fork` 对比：

函数	是否创建新进程	是否替换程序	成功后是否返回
fork	是	否	父子都返回
execve	否	是	不返回

加载流程：

```
删除旧用户虚拟区域
建立新代码段、数据段、堆、栈、共享库映射
把argv和envp放到新栈中
跳到_start开始执行
```

6. `exit` 与 `waitpid`

进程终止后，内核保留其退出状态，直到父进程回收。

```
pid_t waitpid(pid_t pid, int *statusp, int options);
```

常用形式：

```
waitpid(-1, &status, 0);
```

表示等待任意子进程终止。

非阻塞回收常用于信号处理：

```
while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
    // 回收已经终止的子进程
}
```

僵死进程：子进程已经终止，但父进程尚未回收其退出状态。

7. 信号

信号是内核通知进程某个事件发生的一种机制。

常见信号：

信号	含义
SIGINT	终端中断
SIGKILL	强制终止
SIGSEGV	非法内存访问
SIGCHLD	子进程停止或终止
SIGFPE	算术异常，例如除以 0

SIG = signal 信号

INT = interrupt，中断

KILL = kill，强杀

SEGV = segmentation violation，段错误

CHLD = child，子进程

FPE = floating point exception，算术异常

信号处理方式：

- 默认处理。
- 忽略。
- 自定义信号处理函数。

信号处理程序注意事项：

- 尽量短，避免阻塞其他信号处理。
- 尽量只调用异步信号安全函数，常见库函数未必适合在信号处理函数中调用；
- 修改全局变量时使用 `volatile sig_atomic_t` 等方式。
- 需要保护临界区时使用信号阻塞。

8. Shell执行程序全过程

Shell执行外部命令的典型流程：

Shell读取命令

↓

fork创建子进程

↓

子进程execve加载目标程序

↓

目标程序从_start开始执行

↓

main执行程序主体逻辑

↓

程序exit终止

↓

内核向Shell发送SIGCHLD

↓

Shell调用waitpid回收子进程

↓

Shell继续等待下一条命令

9. 前台作业、后台作业与进程组

Shell中：

- 前台作业会占用终端，Shell等待其完成。
- 后台作业不阻塞Shell。
- 终端信号通常发送给前台进程组。

因此，`Ctrl-C` 通常终止前台作业，而不是Shell本身。

第9章 虚拟内存

本章解释虚拟地址如何翻译为物理地址，以及操作系统如何用虚拟内存实现缓存、内存管理和保护。掌握本章后，应能进行地址翻译、解释缺页、理解页表/TLB/mmap/进程地址空间。

1. 虚拟内存的三个作用

作用	解释
----	----

缓存	物理内存作为磁盘中虚拟页的缓存
管理	为每个进程提供独立虚拟地址空间
保护	通过页表权限位控制访问

如果所有进程直接使用物理地址，会出现地址冲突、内存保护困难、程序装载复杂等问题。虚拟内存让每个进程都以为自己拥有完整地址空间。

2. 地址翻译基本概念

CPU生成虚拟地址VA，MMU将其翻译成物理地址PA。

页大小为 $P=2^p$ 时：

```
VPN = VA >> p
VPO = VA & (P - 1)
```

如果页表给出物理页号PPN，则：

```
PP0 = VPO
PA = PPN || PP0 (|| 代表二进制位拼接)
```

核心结论：页内偏移不需要翻译，虚拟页号通过页表翻译成物理页号。

3. 页表与PTE

页表是虚拟页到物理页的映射表。页表条目PTE通常包含：

字段	含义
有效位	该虚拟页是否在物理内存中
PPN	物理页号
权限位	是否可读、可写、可执行、用户可访问
脏位	页面是否被修改过
引用位	页面是否被访问过

如果有效位为1，PPN有效；如果有效位为0，可能是页面未分配，也可能是已分配但不在物理内存中。

4. TLB

TLB是页表项的高速缓存，用来加速地址翻译。

没有TLB时，每次访问内存可能需要先访问页表，再访问数据。TLB保存最近使用的VPN→PPN映射，使大多数地址翻译不必访问页表。

TLB不命中不等于缺页：

- TLB不命中：TLB中没有该映射。
- 页表命中：页表中PTE有效，页面在物理内存。
- 缺页：PTE无效，MMU进入缺页处理流程；内核还要判断地址和权限是否合法，合法时才调入页面，非法时会触发段错误或保护异常。

5. 完整访存流程

```
graph TD
    A[CPU产生虚拟地址] --> B[TLB查找VPN]
    B --> C[TLB命中：得到PPN]
    B --> D[TLB不命中：查页表]
    D --> E[PTE有效：更新TLB，得到PPN]
    D --> F[PTE无效：触发缺页异常；若处理成功，重新执行原指令]
    E --> G[形成物理地址]
    F --> G
    G --> H[访问Cache/主存]
```

6. 缺页处理

缺页是故障类异常。若地址和权限合法，处理完成后，导致缺页的指令会重新执行；若访问非法，进程会被终止或收到相应信号。

处理流程：

MMU发现PTE无效

↓

触发缺页异常，进入内核

↓

判断虚拟地址是否合法

↓

非法：终止进程或发送信号

合法：选择物理页帧

↓

必要时写回牺牲页

↓

从磁盘文件或匿名零页调入目标页

↓

更新PTE，填入PPN和有效位

↓

返回用户程序，重新执行原指令

关键点：PTE中的PPN通常由缺页处理过程在页面调入后填写。

7. 多级页表

以4KB页面为例：

页大小 = 4KB = 2^{12}

页内偏移 = 12位

若每个PTE为8B，一张4KB页表能容纳：

$4096 / 8 = 512 = 2^9$ 个PTE

所以每一级页表索引通常是9位。

多级页表的作用：

- 不为未使用的大量虚拟地址分配页表页。
- 节省页表占用的物理内存。

TLB用于减少多级页表访问带来的性能开销。

8. 进程虚拟地址空间

常见布局：



各区域含义：

区域	内容
.text	程序机器指令，通常只读
.data	已初始化全局变量和静态变量
.bss	未初始化全局变量和静态变量，初始为0
堆	运行时堆区域，了解其在地址空间中的位置即可
共享库	动态链接库映射区域
栈	局部变量、返回地址、参数
内核区	用户程序不能直接访问

易错点：局部变量即使初始化为0，也在栈上，不在 .bss 。

9. 虚拟内存与Cache联合分析

当页大小为4KB时：

$$VP0 = PP0 = \text{低12位}$$

如果Cache的块内偏移和组索引都在低12位内，那么这些位翻译前后不变。但Cache最终仍应理解为用物理地址访问。

联合题步骤：

1. 由页大小拆 VPN/VPO 。
2. 查页表得到 PPN 。
3. 拼出物理地址 PA 。
4. 用物理地址拆Cache的 tag/set/offset 。

10. 页面分布计算

若数组起始页内偏移为 `off` ，数组总字节数为 `n` ，页大小为 `P` ，则占用页数：

```
页数 = ceil((off + n) / P)
```

关键：不能只看数组大小，还要看起始地址是否在页首。

11. 地址翻译

1. 根据页大小确定偏移位数。
2. 拆出VPN和VPO。
3. 查TLB和页表。
4. 得到PPN后拼接物理地址。

易错点

- TLB不命中不等于缺页。
- 缺页处理后重新执行当前指令。
- 页表项里存PPN，不存完整物理地址。
- `mmap` 不是地址翻译表。
- 局部变量不在 `.bss` 。